STAT 387 **HOMEWORK # 3**

Conceptual

1. 3 (a, b; page 220)

3)a) Explain how k-fold cross-validation is implemented:

k-fold cross-validation is implemented by randomly splitting the set of *n* observations into *k* non-overlapping groups, or folds, of approximately equal size. The procedure follows these steps:

1. The model is trained *k* times.
2. In each iteration, a different fold is treated as the validation set, while the remaining *k-1* folds are grouped together to form the training set.
3. The Mean Squared Error (MSE) is calculated for the held-out fold in each iteration.
4. The overall test error estimate is computed by averaging the *k* resulting MSE estimates: $CV_{(k)} = \frac{1}{k}\sum_{i=1}^{k} MSE_i$

3)b)What are the advantages and disadvantages of k-fold cross validation relative to:

i. The validation set approach?

Advantages:

Lower Variability: The validation set approach can be highly variable depending on which observations happen to end up in the training vs. validation sets. k-fold CV provides a more stable and reliable estimate of the test error.

Lower Bias: Because k-fold CV uses a larger fraction of the available data (n(k-1)/k observations) for training in each iteration, it tends to not overestimate the test error as much as the validation set approach does.

Disadvantages:

Computational Cost: k-fold CV is more computationally expensive because it requires fitting the model *k* times, whereas the validation set approach only requires a single fit.

ii. LOOCV?

- **Advantages**:

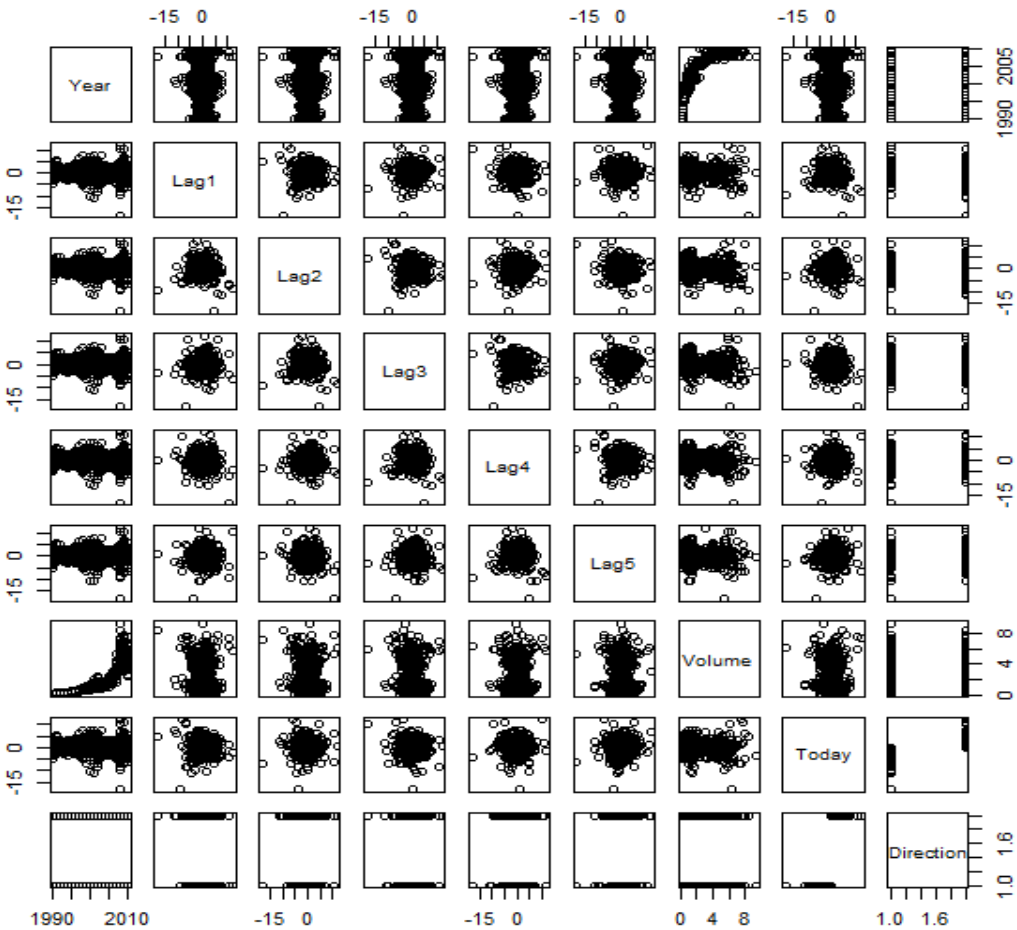
    - Computational Efficiency: If *n* is large, LOOCV is very slow because it requires *n* model fits. k-fold CV (typically with k=5 or k=10) is significantly faster.
    - Lower Variance: In LOOCV, the *n* training sets are almost identical to one another, making the outputs of the models highly correlated. Since the mean of highly correlated quantities has higher variance than the mean of less correlated quantities, LOOCV typically has higher variance in its test error estimate than k-fold CV.
- Disadvantages:
    - Higher Bias: k-fold CV has slightly more bias than LOOCV because LOOCV uses *n-1* observations for training (almost the entire dataset), while k-fold uses slightly fewer.

<u>Applied</u>

2. 13 (a, b, c, d, e, f, g, h, i; page 193)

13)a) Comments: The only substantial pattern is the relationship between Year and Volume. Beyond that, the data appears to be largely random, indicating that there is little to no simple linear relationship between previous weeks' returns and the current week's performance.

```
      Year           Lag1               Lag2               Lag3               Lag4               Lag5
 Min.   :1990   Min.   :-18.1950   Min.   :-18.1950   Min.   :-18.1950   Min.   :-18.1950   Min.   :-18.1950
 1st Qu.:1995   1st Qu.: -1.1540   1st Qu.: -1.1540   1st Qu.: -1.1580   1st Qu.: -1.1580   1st Qu.: -1.1660
 Median :2000   Median :  0.2410   Median :  0.2410   Median :  0.2410   Median :  0.2380   Median :  0.2340
 Mean   :2000   Mean   :  0.1506   Mean   :  0.1511   Mean   :  0.1472   Mean   :  0.1458   Mean   :  0.1399
 3rd Qu.:2005   3rd Qu.:  1.4050   3rd Qu.:  1.4090   3rd Qu.:  1.4090   3rd Qu.:  1.4090   3rd Qu.:  1.4050
 Max.   :2010   Max.   : 12.0260   Max.   : 12.0260   Max.   : 12.0260   Max.   : 12.0260   Max.   : 12.0260
     Volume            Today          Direction
 Min.   :0.08747   Min.   :-18.1950   Down:484
 1st Qu.:0.33202   1st Qu.: -1.1540   Up  :605
 Median :1.00268   Median :  0.2410
 Mean   :1.57462   Mean   :  0.1499
 3rd Qu.:2.05373   3rd Qu.:  1.4050
 Max.   :9.32821   Max.   : 12.0260
```



13)b) Comments: Based on the summary(mod1) output, Lag2 is the only statistically significant predictor, with a p-value of 0.0296. This is below the common 5% significance threshold ($\alpha = 0.05$). All other

predictors, including Volume and the other Lag variables, have p-values well above 0.05, indicating they do not have a clear linear relationship with the probability of the market moving Up or Down in this specific model.

```
Call:
glm(formula = Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 +
    Volume, family = "binomial")

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  0.26686    0.08593   3.106   0.0019 **
Lag1        -0.04127    0.02641  -1.563   0.1181
Lag2         0.05844    0.02686   2.175   0.0296 *
Lag3        -0.01606    0.02666  -0.602   0.5469
Lag4        -0.02779    0.02646  -1.050   0.2937
Lag5        -0.01447    0.02638  -0.549   0.5833
Volume      -0.02274    0.03690  -0.616   0.5377
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 1496.2  on 1088  degrees of freedom
Residual deviance: 1486.4  on 1082  degrees of freedom
AIC: 1500.4

Number of Fisher Scoring iterations: 4
```

13)c) Comments: The diagonal numbers on the table (54 and 557) are where the model predicted correctly, and the mistakes are the off-diagonal numbers(430 and 48). This means that the model predicted the market was "Up" 430 times when the market was actually "Down,"

and it predicted the market was "Down" 48 times when it actually went "Up." This model has a 56.1% accuracy, which is slightly better than a coin flip, showing that the model is better at predicting "Up" days and is biased in that regard. The model notices that the market goes up more than down, so it defaults to that result more often than what is true.

```
Confusion Matrix and Statistics

          Reference
Prediction Down  Up
      Down   54  48
      Up    430 557

               Accuracy : 0.5611
```

13)d)

```
Confusion Matrix and Statistics

          Reference
Prediction Down Up
      Down    9  5
      Up     34 56

               Accuracy : 0.625
```

13)e)

```
Confusion Matrix and Statistics

          Reference
Prediction Down Up
      Down    9  5
      Up     34 56

               Accuracy : 0.625
```

13)f)

```
Confusion Matrix and Statistics

          Reference
Prediction Down Up
      Down    0  0
      Up     43 61

               Accuracy : 0.5865
```

13)g)

```
Confusion Matrix and Statistics

          Reference
Prediction Down Up
      Down   21 30
      Up     22 31

               Accuracy : 0.5
```

13)h)

```
Confusion Matrix and Statistics

          Reference
Prediction Down Up
      Down    0  0
      Up     43 61

               Accuracy : 0.5865
```

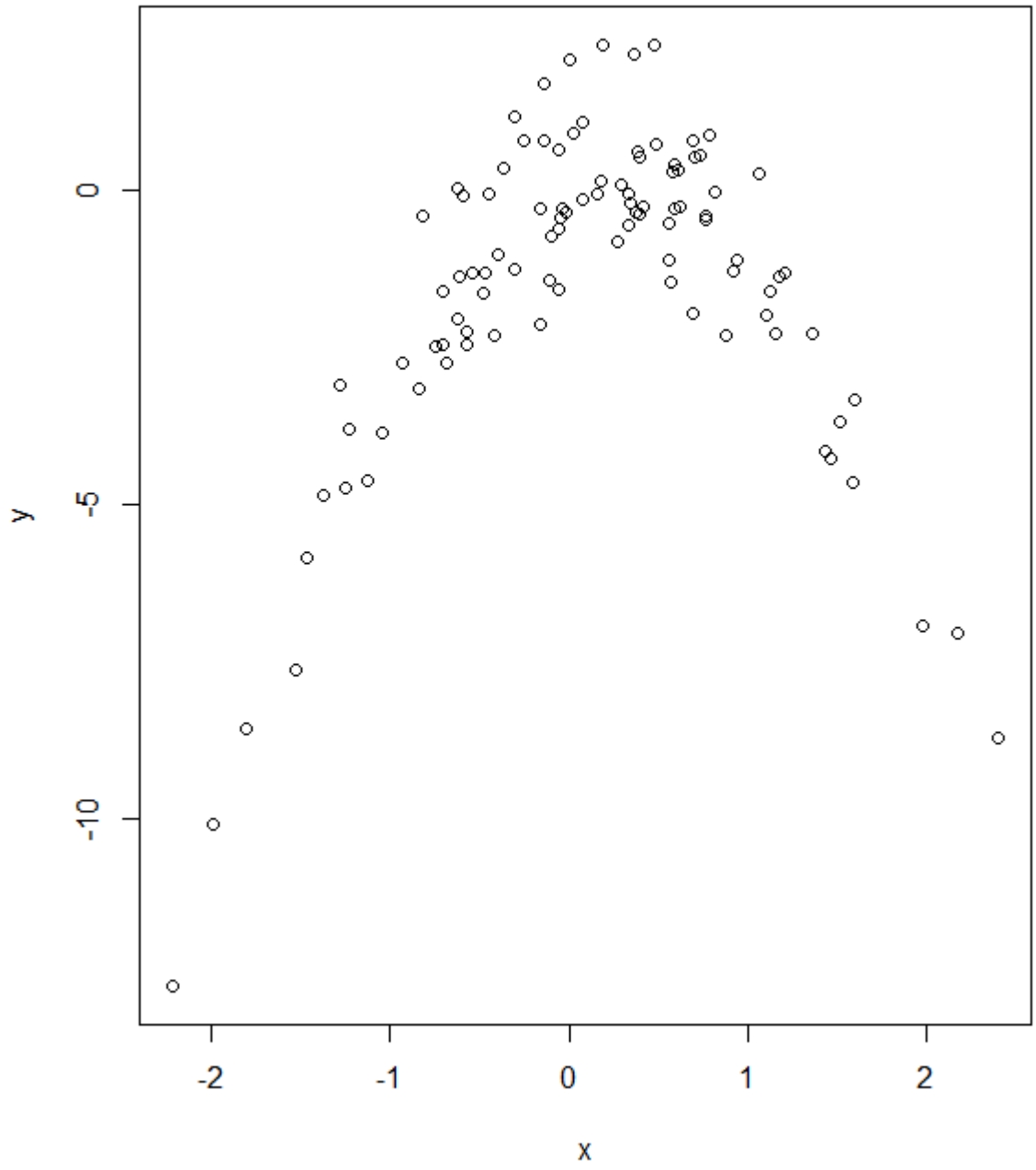
13)i) Based on the results above, Logistic Regression and Linear Discriminant Analysis (LDA) provided the best results for these models with the highest test accuracy of 62.5% for both methods.
While QDA and Naive Bayes produced a slightly lower accuracy of 58.7%, the confusion matrix for these models reveals that they simply predicted “up” for every observation and failed to properly classify “down” weeks in the testing dataset. KNN(k=1) was the least effective

with a 50% accuracy that is no better than a random coinflip. Therefore, the linear methods of Logistic Regression and LDA are superior choices for this dataset because these methods successfully managed to differentiate some downwards trends while maintaining a higher overall success rate and avoiding the simplistic results of QDA and Naive Bayes. The success of the linear model also suggests that the relationship between Lag2 and Direction is more likely linear than quadratic or non-parametric.

3. 8 (a, b, c; pages 222-223)

8)a) In this data set, n = 100 and p = 2. The model used to generate this term is as follows: $y = x - 2x^2 + \varepsilon,\ \varepsilon \sim N(0, 1)$

8)b) Comments: The plot below shows a distinct upside down U shaped pattern that is indicative of a non-linear relationship between x and y, more akin to a quadratic pattern. The negative term in the equation gives rise to the upside U-shape.


0
-5
-10
y
-2
-1
0
1
2
x

8)c)LOOCV errors with a random seed set to 1 are as follows**:**

```
7.2881616 0.9374236 0.9566218 0.9539049
```

R-Code

Question 13)(on next page)

```r
library(ISLR2)
library(glmnet)
library(caret)
attach(Weekly)
#a)Produce some numerical and graphical summaries of the Weekly
#data. Do there appear to be any patterns?
summary(Weekly)
pairs(Weekly)
#b)Use the full data set to perform a logistic regression with
#Direction as the response and the five lag variables plus Volume
#as predictors. Use the summary function to print the results.
mod1 <- glm(Direction ~ Lag1+Lag2+Lag3+Lag4+Lag5+Volume, data = Weekly, family="binomial")
summary(mod1)
#(c) Compute the confusion matrix and overall fraction of correct
#predictions.
modProbs <- predict(mod1, type = "response")
modPred <- ifelse(modProbs > 0.5, "Up", "Down")
table(modPred, Weekly$Direction)
mean(modPred == Weekly$Direction)
actualDirection <- Weekly$Direction
predsFactor <- factor(modPred, levels = c("Down", "Up"))
actualDirectionFactor <- factor(actualDirection, levels = c("Down", "Up"))
confusionMatrix(predsFactor, actualDirectionFactor)
#(d) Now fit the logistic regression model using a training data period
#from 1990 to 2008, with Lag2 as the only predictor. Compute the
#confusion matrix and the overall fraction of correct predictions
#for the held out data (that is, the data from 2009 and 2010).
train <- (Weekly$Year < 2009)
mod2 <- glm(Direction ~ Lag2, data = Weekly, family = binomial, subset = train)
mod2Prob <- predict(mod2, Weekly[!train, ], type = "response")
mod2Pred <- ifelse(mod2Prob > 0.5, "Up", "Down")
table(mod2Pred, Weekly$Direction[!train])
mean(mod2Pred == Weekly$Direction[!train])
predsFactor2 <- factor(mod2Pred, levels = c("Down", "Up"))
actualDirectionFactor2 <- factor(Weekly$Direction[!train], levels = c("Down", "Up"))
confusionMatrix(predsFactor2, actualDirectionFactor2)

#(e) Repeat (d) using LDA.
library(MASS)
lda.fit <- lda(Direction ~ Lag2, data = Weekly, subset = train)
lda.pred <- predict(lda.fit, Weekly[!train, ])
table(lda.pred$class, Weekly$Direction[!train])
mean(lda.pred$class == Weekly$Direction[!train])
predsFactorLDA <- factor(lda.pred$class, levels = c("Down", "Up"))
actualDirectionFactorLDA <- factor(Weekly$Direction[!train], levels = c("Down", "Up"))
confusionMatrix(predsFactorLDA, actualDirectionFactorLDA)

#(f) Repeat (d) using QDA.
qda.fit <- qda(Direction ~ Lag2, data = Weekly, subset = train)
qda.pred <- predict(qda.fit, Weekly[!train, ])
table(qda.pred$class, Weekly$Direction[!train])
mean(qda.pred$class == Weekly$Direction[!train])
predsFactorQDA <- factor(qda.pred$class, levels = c("Down", "Up"))
actualDirectionFactorQDA <- factor(Weekly$Direction[!train], levels = c("Down", "Up"))
confusionMatrix(predsFactorQDA, actualDirectionFactorQDA)

#(g) Repeat (d) using KNN with K = 1.
library(class)
train.X <- as.matrix(Weekly$Lag2[train])
test.X <- as.matrix(Weekly$Lag2[!train])
train.Direction <- Weekly$Direction[train]
set.seed(1)
knn.pred <- knn(train.X, test.X, train.Direction, k = 1)
table(knn.pred, Weekly$Direction[!train])
mean(knn.pred == Weekly$Direction[!train])
predsFactorKNN <- factor(knn.pred, levels = c("Down", "Up"))
actualDirectionFactorKNN <- factor(Weekly$Direction[!train], levels = c("Down", "Up"))
confusionMatrix(predsFactorKNN, actualDirectionFactorKNN)

#(h) Repeat (d) using naive Bayes
library(e1071)
nb.fit <- naiveBayes(Direction ~ Lag2, data = Weekly, subset = train)
nb.pred <- predict(nb.fit, Weekly[!train, ])
table(nb.pred, Weekly$Direction[!train])
mean(nb.pred == Weekly$Direction[!train])
predsFactorNB <- factor(nb.pred, levels = c("Down", "Up"))
actualDirectionFactorNB <- factor(Weekly$Direction[!train], levels = c("Down", "Up"))
confusionMatrix(predsFactorNB, actualDirectionFactorNB)
```

Question 8)

```r
#Question 8
#8)a) In this dataset what is n and what is p? Write out the model used to generate the data in equation form.
set.seed(1)
x <- rnorm(100)
y <- x - 2 * (x^2) + rnorm(100)
n <- length(y) # n = 100
p <- 2 # p = 2 (x and x^2)
n
p
# Model: y = β0 + β1*x + β2*x^2 + ϵ
plot(x, y)

#c)set a random seed, and then compute the LOOCV errors that result from fitting the following four models:
#i) Y = β0 + ϵ
#ii) Y = β0 + β1X + ϵ
#iii) Y = β0 + β1X + β2X^2 + ϵ
#iv) Y = β0 + β1X + β2X^2 + β3X^3 + ϵ
set.seed(1)
loocv_errors <- sapply(1:4, function(degree) {
  cv_error <- 0
  for (i in 1:n) {
    train_x <- x[-i]
    train_y <- y[-i]
    test_x <- x[i]
    test_y <- y[i]
    model <- lm(train_y ~ poly(train_x, degree))
    pred <- predict(model, newdata = data.frame(train_x = test_x))
    cv_error <- cv_error + (pred - test_y)^2
  }
  return(cv_error / n)
})
names(loocv_errors) <- c("Degree 0", "Degree 1", "Degree 2", "Degree 3")
loocv_errors
```